

Implementation Overview

The implementation of the project closely follows the class diagrams and design plan that I created in Stage 1. The system uses 5 main components:

1. User: owns multiple categories, each with tasks and a plant.
2. Category: groups tasks and contains a plant.
3. Task: individual tasks the user can create, edit, and complete.
4. Plant: visual growth progression based on completed tasks.
5. StorageManager (Singleton Pattern): manages JSON persistence across the entire system.

Although the original Stage 1 plan was to develop a full mobile application, I implemented a command-line interface (CLI) version instead, focusing on core functionality and testing to ensure a robust and functional system.

Key Programming Principles Applied

Object-Oriented Programming:

Encapsulation: Each class manages its own data and behavior.

Composition: Users contain categories; categories contain tasks and a plant.

Abstraction: The controller hides all the programming details from the user (the user doesn't see the code, just the output).

Modular Code Structure:

Code is divided into the folders models/, controller/, and utils/, improving maintainability and readability.

Documentation:

Each class and method includes docstrings describing purpose and behavior.

System Requirements Fulfillment

Interrelated Classes:

TaskGarden implements 5 fully connected classes: User, Category, Task, Plant, StorageManager

Component Interaction:

The program uses a simple MVC structure:

Model: User, Category, Task, Plant

Controller: TaskGardenController

View: CLI menus and printed output

Error Handling:

Duplicate categories rejected with an exception. Completing an already completed task raises ValueError. Input validation for menu selections and category choices.

Design Pattern Requirement:

TaskGarden uses the Singleton pattern in StorageManager, ensuring that only one instance manages JSON data and all components refer to the same storage state.

Testing and Validation

Unit Test: Using Python's unittest framework, I created tests for plant growth behavior, task completion rules, category task storage, user category management, and singleton behavior of storage manager.

```
test 3 - add task to category - passed!  
test 4 - category to/from dict - passed!  
test 1 - plant growth - passed!  
test 7 - storage singleton - passed!  
test 2 - task completion - passed!  
test 5 - add category - passed!  
test 6 - user to/from dict - passed!
```

```
-----  
Ran 7 tests in 0.002s
```

Manual Testing: I used manual testing to demonstrate adding categories and tasks, completing tasks and observing plant growth, navigating menus, and to verify the updated JSON file structure after each action.

1. View Garden
2. View Tasks
3. Add Category
4. Add Task
5. Complete Task
6. Exit

Choose an option:

1. Login
2. Register
3. Exit

Select option:

```
{
  "users": [
    {
      "username": "123",
      "password_hash": "a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3",
      "categories": [
        {
          "name": "chores",
          "tasks": [
            {
              "name": "do dishes",
              "completed": true,
              "completed_date": "2025-11-19T17:11:18.786069"
            },
            {
              "name": "sweep",
              "completed": true,
              "completed_date": "2025-11-19T17:13:33.496802"
            }
          ],
          "plant": {
            "species": "fern",
            "growth_level": 2
          }
        }
      ]
    }
  ],
}
```